

Doc. no.: P0803R0
Date: 2017-10-15
Reply to: Beman Dawes <bdawes at acm dot org>
Audience: Library Evolution

Endian Library Request for Comments (R0)

Abstract

The Boost Endian library manipulates the [endianness](#) of integers and user-defined types. It provides three approaches to dealing with endianness: conversion functions, buffer classes, and arithmetic classes. Each approach has use cases where it is preferred over the other approaches. The purpose of this RFC is to determine interest in adding all or any of these components to the C++ standard library or a library TS.

Introduction

The Boost Endian documentation (www.boost.org/libs/endian/doc) provides an [overview](#) and detailed technical specifications for the [conversion functions](#), [arithmetic types](#), and [buffer types](#).

A separate discussion covers [choosing between the three approaches](#) provided by the library.

For those not interested in plowing through the entire documentation, the [Overall FAQ](#), [Conversion FAQ](#), [Arithmetic FAQ](#), and [Buffers FAQ](#) answer a lot of question likely to arise during standardization.

Motivation for standardization

- Lack of any one de facto standard endianness capability in either C or C++. While most platforms provides some way to handle endianness in C, the mechanism varies from platform to platform.
- Commonly used endian handling functions are extremely error prone. While the Boost Endian conversion functions have the same problems, the buffer and arithmetic classes do not.
- Surprisingly difficult to write and test yourself in a portable yet efficient (i.e. using intrinsics) way.
- Explicit requests for standardization from Boost Endian users.
- The Boost library is existing practice with years of both implementation and end-user experience.
- The committee has already failed once to standardize endian conversion functions; perhaps now is time to do it right.

Impact on the standard library

All three proposed approaches to endianness would be a pure additions to the standard library and would break no existing user code (modulo the usual namespace caveats).

Proposed wording

The Boost Endian reference documentation for [Conversion](#), [Arithmetic](#), and [Buffer](#) approaches follows the standard library's *Method of description* [description], so provides a fairly close approximation of what proposed wording would look like. Actual proposed wording will be provided in a proposal document for any portions of the library of interest to the committee. Names will need the usual bikeshedding.

Questions for the Library Evolution Group

- How would you likely vote on a fully-worded and reviewed proposal to add a standardized version of the Boost Endian conversion functions to the standard library or a library TS?
- How would you likely vote on a fully-worded and reviewed proposal to add a standardized version of the Boost Endian buffer classes to the standard library or a library TS?
- How would you likely vote on a fully-worded and reviewed proposal to add a standardized version of the Boost Endian arithmetic classes to the standard library or a library TS?

Acknowledgements

The original design for the arithmetic classes was developed by Darin Adler based on work by Mark Borgerding. The Boost documentation acknowledges [45+ other people](#).
